

Clean Code - Chapter 5: Formatting (Complete Notes)

Purpose of Formatting

Formatting is about communication. Readability is more important than functionality because code changes over time, but style and readability persist.

Vertical Formatting

Keep files small (preferably under 200 lines). Smaller files are easier to understand.

Newspaper Metaphor

Code should be organized like a newspaper: high-level concepts at the top, details progressively below.

Vertical Openness Between Concepts

Use blank lines to separate different concepts.

Bad Example

```
class A{int x;int y;void f(){do1();do2();}}
```

Good Example

```
class A {  
    int x;  
    int y;  
  
    void f() {  
        do1();  
        do2();  
    }  
}
```

Vertical Density

Lines that are closely related should be kept together without unnecessary comments.

Bad Example

```
private String name;  
/* comment */  
private List items;
```

Good Example

```
private String name;  
private List items;
```

Vertical Distance

Related concepts should be placed close together.

Variable Declarations

Declare variables close to where they are used. Instance variables should be at the top.

Dependent Functions

Caller functions should appear above the functions they call.

Bad Example

```
void a(){b();}  
void c(){}  
void b(){}
```

Good Example

```
void a(){b();}  
void b(){}
```

Conceptual Affinity

Functions that do similar things should be placed close together.

Vertical Ordering

High-level functions should come first, low-level details later.

Horizontal Formatting

Keep line length under ~120 characters.

Horizontal Openness and Density

Use spaces to separate elements. Do not separate function names and parentheses.

Bad Example

```
return(-b+Math.sqrt(d))/(2*a);
```

Good Example

```
return (-b + Math.sqrt(d)) / (2*a);
```

Operator Precedence

High-precedence operators may not need spaces; low-precedence operators should have spaces.

```
return b*b - 4*a*c;
```

Horizontal Alignment

Avoid aligning variables vertically. It reduces readability.

Indentation

Indentation shows structure. Without it, code becomes unreadable.

Bad Example

```
if(x){if(y){do();}}
```

Good Example

```
if (x) {  
    if (y) {  
        do();  
    }  
}
```

Avoid Breaking Indentation

Do not collapse blocks into one line.

Dummy Scopes

Avoid empty loop bodies. If used, make them visible.

```
while(condition)  
;
```

Team Rules

Teams should agree on one formatting style and follow it consistently.

Key Principles

- Formatting = communication - Small, structured files - Clear separation of concepts - Proper spacing and indentation - Consistency across the team